Язык программирования Python

Классы

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Синтаксис объявления класса

Объявить класс можно, например, следующим образом:

Заголовок класса. Имя обычно с большой буквы

```
class Имя_класса:
    """ Описание класса """

    # Конструктор. Инициализирует поля, значения
    # которых доступны только одному конкретному
    # экземпляру. self хранит этот экземпляр.
    def __init__(self, параметр1):
        # Полей может быть несколько
        self.поле1 = параметр1

    другие_методы_класса
```

Выводится, если использовать функцию help()

Объявляются как обычные функции, принимающие первым параметром self

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

В теле класса могут быть

Поля – переменные, хранящие информацию (состояние) конкретного экземпляра класса.

Методы:

- **Методы** – функции, получающие в виде первого параметра конкретный экземпляр класса (обозначают идентификатором `self`) и реализующие для него какие-то вычисления или другие действия.
- **Конструктор** – специальный метод, реализующий действия по инициализации полей экземпляров класса. Имеет имя `__init__`. В конструкторе также происходит добавление всех полей экземпляров класса. Вызывается автоматически один раз при создании экземпляра.
- **Операции** – задают действия с объектами с помощью знаков операций. Также имеют заданные имена (например, `__add__` для сложения, `__sub__` для вычитания и т.д.).
- **Другие предопределённые методы** – это ряд методов, для которых предусмотрены специальные имена. Например, метод `__str__`, возвращающий строковое представление объекта.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса

```
class Box:
    """Класс является моделью коробки"""

    # В self хранится только что созданный экземпляр класса,
    # в других параметрах - значения полей этого экземпляра
    def __init__(self, length, width, height):
        # Создаём поля, которые будут у экземпляров класса
        self.length = length # длина
        self.width = width   # ширина
        self.height = height # высота

    def square(self):
        """Метод возвращает площадь основания"""
        return self.length * self.width

    def volume(self):
        """Метод возвращает объём коробки"""
        # В одном методе можно вызвать другой метод
        return  self.square() * self.height
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис создания экземпляра (1)

Экземпляр (объект) класса – это переменная типа класса.

Создаётся он следующим образом:

имя_экземпляра = имя_класса (параметры_из_конструктора)

все, кроме self

Например, можно создать экземпляр класса **Box** с длиной 3, шириной 4 и высотой 2:

```
# Создаётся экземпляр класса Box.
d = Box(3, 4, 2) # При этом вызывается метод __init__
```

Стоит отметить, что если в конструкторе не задать параметрам значения по умолчанию, то для создания экземпляров всегда нужно будет указывать их все:

```
e = Box()
TypeError: __init__() missing 3 required positional arguments: 'length', 'width', and 'height'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис создания экземпляра (2)

Для решения проблемы желательно всегда присваивать параметрам конструктора значения по умолчанию. Это могут быть какие-нибудь конкретные строковые или числовые значения, а **может быть значение None**.

Перепишем объявление класса Box и его конструктор (оставшиеся два метода неизменны):

0 является значением по умолчанию, т.е. если не указать аргумент

```
class Box:
    def __init__(self, length=0, width=0, height=0):
        # Полям будут присвоены значения аргументов
        self.length = length # длина
        self.width = width   # ширина
        self.height = height # высота
```

Создать экземпляр теперь можно так:

```
e = Box()
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис вызова метода

Вызов метода класса осуществляется с помощью следующей конструкций:

все, кроме self

имя_экземпляра . имя_метода(параметры_метода)

Например, вызов метода **volume**() для объекта **d** может быть записан так:

```
d = Box(3, 4, 2)
vol = d.volume()
print(f'Объём коробки d = {vol}') # Выведет 24 (3*4*2)
```

Для доступа к полям конкретного экземпляра используется конструкция:

имя_экземпляра . имя_поля

Например, можно работать напрямую с полем **height** объекта **d**:

```
print(d.height) # выведет 2
d.width = 3.5 # присвоит полю width значение 3.5
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Замечания по доступу к полям

Хорошим тоном считается, что для чтения/изменения какого-либо поля должны использоваться специальные методы, которые называются getter/setter. Эти методы, как правило, имеют следующие имена:

get_имя_поля() # возвращает значение поля
set_имя_поля(значение) # устанавливает значение поля

При этом сами **имена полей начинаются с двух символов подчёркивания**. В этом случае обратиться к ним извне класса нельзя.

Делается такое усложнение для того, чтобы можно было добавить проверку корректности значений, присваиваемых полям, или скорректировать вывод.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример доступа к полям через методы

```
# Описание класса
class Circle:
    """Класс является моделью круга"""

    def __init__(self):
        # добавляем поле, значение устанавливаем по умолчанию
        self.__radius = 0

    def get_radius(self): # Метод возвращает радиус
        return self.__radius

    def set_radius(self, radius): #Метод устанавливает радиус
        self.__radius = 0 if (radius < 0) else radius
```

```
# Основная программа
cr = Circle()
r = float(input('Введите радиус: '))
cr.set_radius(r)
print(cr.get_radius())
```

```
In [10]: runfile('D
wdir='D:/Sources/_J

Введите радиус: 6
6.0

In [11]: runfile('D
wdir='D:/Sources/_J

Введите радиус: -2
0
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса из лабораторной 1 (1)

Методы доступа нужно делать только для тех полей, к которым действительно нужен доступ. В ряде случаев его можно заменить на методы проверки значений полей переданным параметрам.

Например, необходимо разработать класс TV, моделирующий телевизор. Он будет иметь следующие поля: Название, Диагональ, Цена, Фирма.

Реализуем методы для заполнения полей с клавиатуры и вывода на экран.

Также напишем методы для проверки того, выпускается ли телевизор указанной фирмой и находится ли цена в указанном диапазоне (эти методы должны возвращать True или False).

```
# Описание класса TV, моделирующего телевизор
class TV:
    # Значения по умолчанию позволяют создавать экземпляры
    # класса без передачи параметров
    def __init__(self, name = None, brand = None,
                 diag = None, price = None):
        self.__name = name
        self.__brand = brand
        self.__diag = diag
        self.__price = price
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса из лабораторной 1 (2)

```
# Метод заполняет поля с клавиатуры
def input_from_keyboard(self):
    self.__name = input('Введите название телевизора: ')
    self.__brand = input('Введите название фирмы: ')
    self.__diag = float(input('Введите диагональ телевиз.: '))
    self.__price = float(input('Введите цену телевизора: '))
    return self # Возвращаем в качестве результата заполненный

# Метод вывода на экран значений полей
def print_to_display(self):
    print(f'Телевизор {self.__name} компании {self.__brand} '
          f'диагональю {self.__diag} и '
          f'стоимостью {self.__price} руб.')

# Проверяем, совпадает ли значение в поле __brand со значением
# переданного параметра brand
def is_this_brand(self, brand):
    # Перед сравнением приводим всё к нижнему регистру
    return self.__brand.lower() == brand.lower()

def is_in_this_price_range(self, low, high):
    return low <= self.__price <= high
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса из лабораторной 1 (3)

```
# Основная программа. Оформим как функцию main()

def main():
    # Будем заполнять список телевизоров
    # Для этого запрашиваем количество телевизоров
    num = int(input('Сколько телевизоров вводите? '))

    # В список tvs будем сохранять объекты класса TV
    tvs = []

    # Заданное количество раз выполняем цикл
    for _ in range(num):
        tv = TV()                  # Создаём экземпляр
        tv.input_from_keyboard() # Заполняем его поля
        tvs.append(tv)             # Добавляем в список

    # Выводим все телевизоры на экран
    for tv in tvs:
        tv.print_to_display()
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса из лабораторной 1 (4)

```
# Продолжение функции main()

# Здесь будем выполнять проверку вхождения в диапазон цен
# Сначала запрашиваем от пользователя данные для сравнения
min_price = float(input('Какая минимальная цена? '))
max_price = float(input('Какая максимальная цена? '))

# В список tv_price будем сохранять подходящие объекты
tv_price = []

# Перебираем все телевизоры
for tv in tvs:
    # Проверяем, имеет ли цену из указанного диапазона
    if tv.is_in_this_price_range(min_price, max_price):
        tv_price.append(tv)      # Добавляем в список

# Выводим список tv_price на экран
for tv in tv_price:
    tv.print_to_display()
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса из лабораторной 1 (5)

```
    # Продолжение функции main()

    # Здесь будем выполнять проверку фирмы
    # Сначала запрашиваем от пользователя данные для сравнения
    brand = input('Какую фирму ищете? ')

    # В отдельный список заносим телевизоры указанной фирмы
    # При этом используется список включений
    tv_brand = [tv for tv in tvs if tv.is_this_brand(brand)]

    # Выводим полученный список tv_brand на экран
    for tv in tv_brand:
        tv.print_to_display()


# Выполняем функцию main()
if __name__ == '__main__':
    main()
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Результат работы

```
Сколько телевизоров вводите? 3

Введите название телевизора: LED
Sharp LC65UI7252E черный

Введите название фирмы: Sharp

Введите диагональ телевизора: 65

Введите цену телевизора: 55999

Введите название телевизора: LED
Samsung QE82Q900R черный

Введите название фирмы: Samsung

Введите диагональ телевизора: 82

Введите цену телевизора: 799999

Введите название телевизора: LED
Harper 20R470T черный

Введите название фирмы: Harper

Введите диагональ телевизора: 20

Введите цену телевизора: 5199
```

```
Телевизор LED Sharp LC65UI7252E
черный компании Sharp диагональю 65.0
и стоимостью 55999.0 руб.
Телевизор LED Samsung QE82Q900R
черный компании Samsung диагональю
82.0 и стоимостью 799999.0 руб.
Телевизор LED Harper 20R470T черный
компании Harper диагональю 20.0 и
стоимостью 5199.0 руб.

Какая минимальная цена? 5000

Какая максимальная цена? 60000
Телевизор LED Sharp LC65UI7252E
черный компании Sharp диагональю 65.0
и стоимостью 55999.0 руб.
Телевизор LED Harper 20R470T черный
компании Harper диагональю 20.0 и
стоимостью 5199.0 руб.

Какую фирму ищете? Samsung
Телевизор LED Samsung QE82Q900R
черный компании Samsung диагональю
82.0 и стоимостью 799999.0 руб.
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса из лабораторной 2 (1)

Помимо создания класса TV, моделирующего телевизор, также можно написать класс TV_list, назначением которого будет обработка списков экземпляров класса TV.

В этом случае из основной программы можно перенести весь код по работе со списком телевизоров в отдельные методы класса TV_list. Класс TV оставим без изменений.

```
# Описание класса TV_list, моделирующего список телевизор
class TV_list:
    # Для инициализации в конструктор можно передать список
    # экземпляров класса TV
    def __init__(self, tvs = None):
        # Проверяем, передан ли список (isinstance вернёт True,
        # если переменная tvs принадлежит классу list)
        if isinstance(tvs, list):
            self.__tvs = tvs
        # Если нет, то создаём пустой список
        else:
            self.__tvs = []
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса из лабораторной 2 (2)

```
# Метод заполнения списка телевизоров
 def input_from_keyboard(self):
     num = int(input('Сколько телевизоров вводите? '))
     for i in range(num):
         tv = TV().input_from_keyboard()
         self.__tvs.append(tv)

 # Метод вывода списка телевизоров на экран
 def print_to_display(self):
     for tv in self.__tvs: # Выводим все телевизоры на экран
         tv.print_to_display()

 # Метод возвращает новый экземпляр класса TV_list, включающий
 # только те телевизоры, которые удовлетворяют условию
 def get_tvs_of_this_brand(self, brand):
     # Создаём список телевизоров путём выбора необходимых
     result = [tv for tv in self.__tvs
               if tv.is_this_brand(brand)]
     # Преобразуем его в экземпляр класса TV_list
     return TV_list(result)
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса из лабораторной 2 (3)

```
    def get_tvs_in_this_price_range(self, low, high):
        result = [tv for tv in self.__tvs
                  if tv.is_in_this_price_range(low, high)]
        return TV_list(result)

# Основная программа
# Создаём объект класса TV_list
tvs = TV_list()
# Заполняем список телевизоров
tvs.input_from_keyboard()

# Выводим все телевизоры на экран
tvs.print_to_display()

brand = input('Какую фирму ищете? ')
# Выбираем все телевизоры определённой фирмы
tv_brand = tvs.get_tvs_of_this_brand(brand)

# Выводим их на экран
tv_brand.print_to_display()
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса из лабораторной 2 (4)

```
min_price = float(input('Какая минимальная цена? '))
max_price = float(input('Какая максимальная цена? '))
# Выбираем все телевизоры, удовлетворяющие условию вхождения в
# диапазон цен
tv_price = tvs.get_tvs_in_this_price_range(min_price, max_price)

# Выводим их на экран
tv_price.print_to_display()
```

Результат работы этой программы будет таким же, как и на слайде 23:

```
Сколько телевизоров вводите? 3

Введите название телевизора: LED
Sharp LC65UI7252E черный

Введите название фирмы: Sharp

Введите диагональ телевизора: 65

Введите цену телевизора: 55999

Введите название телевизора: LED
Samsung QE82Q900R черный

Введите название фирмы: Samsung

Введите диагональ телевизора: 82

Введите цену телевизора: 799999

Введите название телевизора: LED
Harper 20R470T черный

Введите название фирмы: Harper

Введите диагональ телевизора: 20

Введите цену телевизора: 5199
```

```
Телевизор LED Sharp LC65UI7252E
черный компании Sharp диагональю 65.0
и стоимостью 55999.0 руб.
Телевизор LED Samsung QE82Q900R
черный компании Samsung диагональю
82.0 и стоимостью 799999.0 руб.
Телевизор LED Harper 20R470T черный
компании Harper диагональю 20.0 и
стоимостью 5199.0 руб.

Какая минимальная цена? 5000

Какая максимальная цена? 60000
Телевизор LED Sharp LC65UI7252E
черный компании Sharp диагональю 65.0
и стоимостью 55999.0 руб.
Телевизор LED Harper 20R470T черный
компании Harper диагональю 20.0 и
стоимостью 5199.0 руб.

Какую фирму ищете? Samsung
Телевизор LED Samsung QE82Q900R
черный компании Samsung диагональю
82.0 и стоимостью 799999.0 руб.
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева